LAB 4 (Part#1)
Secondary Data Structures
[CO1]

Instructions for students:

- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python Template**.

NOTE:

- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

`Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited`

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 4 [Part#1] Assignment Tasks: 2
Total Marks: 10

Lab Tasks [No Need to Submit]

1. Hashtable: Insert using Forward Chaining:

Write the **insert()** function and **hash_function()** of the HashTable class which uses an array to store a key-value pair, where the key is a string representing a fruit, and the value is an int representing its respective price.

hash_function(key)

Takes a key which is a string, and calculates its length. If length is even, the sum of the ascii values of the even characters is calculated, otherwise, the sum of the ascii values of odd characters is calculated.
Finally, it returns the sum modded by length of the array

If we call __hash_function("apple"):
As len("apple") is odd, characters in odd indexes (p, l) are taken:

sum = ord(p) + ord(l) = 112 + 108 = 220

If the Hashtable object is initialized with length 5, then:

sum % length of Hashtable array = 220 % 5 = 0
So, the function returns 0

insert(key, value)

Creates a node that contains **a *tuple(for python)* or *an array(for java)*** which stores the key-value pair in 0,1 index respectively. Finds index by passing key to hash_function(). If there is no collision, the node is placed in the index.

If there is a collision, forward chaining is applied. Here, the chain should be arranged in **descending order**, i.e.: if the node being inserted has the highest value (price), it will be the head of the chain. Otherwise, you should iterate the chain and insert it in the appropriate position.

[If the same key is given twice for insertion, update the value of that particular key. No need to create new Node]

Assignment Tasks [Need to Submit]

1. Hashtable: Deletion Operation :

You are given the hash function, **h(key) = (key + 3) % 6** for a hash-table of length 6. In this hashing, forward chaining is used for resolving conflict and a 6-length array of singly linked lists is used as the hash-table. In the singly linked list, each node has a next pointer, an Integer key and a string value, for example: (4 (key) , “Rafi” (value)). The hash-table stores this key-value pair.

Implement a function **remove(hashTable, key)** that takes a key and a hash-table as parameters. The function removes the key-value pair from the aforementioned hashtable if such a key-value pair (whose key matches the key passed as argument) exists in the hashtable.

Consider,Node class and hashTable are given to you. You just need to complete the remove(hashTable, key) function.

```
class Node:
  def __init__(self, key, value, next=None):
      self.key, self.value, self.next = key, value, next
```

Sample Input and Output:
Some Key-value pairs:
(34, “Abid”) , (4, “Rafi”), (6, “Karim”), (3, “Chitra”), (22, “Nilu”)

HashTable is given in the following:
0: (3, “Chitra”)
1: (22, “Nilu”) → (4, “Rafi”) → (34, “Abid”)
2: None
3: (6, “Karim”)
4: None
5: None

remove(hashTable, key=4) returns the changed hashTable where (4, “Rafi”) is removed.

New HashTable Output:
0: (3, “Chitra”)
1: (22, “Nilu”) → (34, “Abid”)
2: None
3: (6, “Karim”)
4: None
5: None

remove(hashTable, key=9) returns the same given hashTable since 9 doesn’t exist in the table.

2. Searching in hashtable :

Complete the **`hashFunction()`** and **`searchHashtable()`** methods. **Do not** change the given code; implement only the required methods. Creating and Inserting into a hash table using forward chaining is already done inside of the class. Do not initialize any other instance variable other than the given ones.

a. **search_hashtable()** → this instance method takes a **key value pair** in the parameter then uses the string key to search for the string in the instance variable **ht**. If the **`string s`** is found, this method returns **'Found'**, else returns **'Not Found'**.
[Do not implement sequential search, implement the hash based search]

b. **hashFunction()** → this instance method takes a key-value pair (string, int), calculates the hashed index on key and returns the index. This hash function takes consecutive two letters of the key string, concatenates their ascii values into an integer and sums all the concatenated integers. Then it finds out the modulus of the summation (**think for yourself with which number should we mod the summation**) as the hashed index.
For instance, for a string 'Mortis', the consecutive two letters are Mo, rt, is.
The concatenated integer for
Mo is 77111 (Ascii of M is 77, o is 111);
rt is 114116 (Ascii of r is 114, t is 116);
'is' is 105115 (Ascii of i is 105, s is 115).
The summation is = 77111+114116+105115
Mod the summation with ___ and return the answer as the hashed index.
[**`fill in the ___ gap with your knowledge`**]

Note: For an odd length string, add the letter 'N' at the end of it. Thus, 'Morti' becomes 'MortiN' and the consecutive two letters are Mo, rt, iN.